

Traffic Signal Controller Chip Project

A Complete Beginner's End-to-End ASIC Design Workflow Guide

Target Architecture: Generic ASIC (Gate-Level Netlist Standard Cells)

Language: Verilog HDL (IEEE 1364-2001)

Platform: EDA Playground Simulation & Verification environment

Document Version: 1.0.0 (Production-Ready)

1. Introduction & The ASIC Design Workflow

Designing an Application-Specific Integrated Circuit (ASIC) represents the pinnacle of hardware customization. Unlike programming software or configuring an FPGA (where hardware resources are re-programmable), an ASIC flow targets the creation of physical silicon where logic gates are permanently routed. This document provides a foundational guide for beginners to design, verify, and conceptually synthesize a 4-State Finite State Machine (FSM) Traffic Light Controller.

The Standard ASIC Flow (Front-End to Back-End)

1. **Specification:** Defining features, truth tables, structural timing, and precise pinouts.
2. **Architectural Design:** Creating state-transition logic, determining structural blocks (counters, sequential registers).
3. **RTL Design (Coding):** Implementing the architecture using a Hardware Description Language (HDL) such as Verilog.
4. **Functional Verification:** Building an automated testbench to drive stimuli into the model and confirm functional accuracy.
5. **Logic Synthesis:** Translation of abstract HDL into physical primitives (AND, OR, D-FlipFlops) via toolsets like Yosys or Synopsys Design Compiler.
6. **Physical Design:** Floorplanning, Placement, Routing, Clock Tree Synthesis (CTS), and physical verification (DRC/LVS).

2. System Specification & Architectural Design

Our traffic system manages a classic four-way intersection split into **Main Street** (high priority, longer green cycle) and **Side Street** (lower priority, shorter green cycle). Each street operates three individual illumination units: Red, Yellow, and Green.

FSM State Mapping & State Table

The control block transitions sequentially through four distinct states based on an internal timer module. The bitwise outputs map to a 3-bit structure where **Light = [Green, Yellow, Red]**. For example, 3'b100 is Green, 3'b010 is Yellow, and 3'b001 is Red.

State ID	State Name	Main Street Output	Side Street Output	Target Duration
2'b00	S0: Main Green	3'b100 (Green)	3'b001 (Red)	5 Clock Cycles
2'b01	S1: Main Yellow	3'b010 (Yellow)	3'b001 (Red)	2 Clock Cycles
2'b10	S2: Side Green	3'b001 (Red)	3'b100 (Green)	4 Clock Cycles
2'b11	S3: Side Yellow	3'b001 (Red)	3'b010 (Yellow)	2 Clock Cycles

3. Verilog RTL Implementation

Save the following source code into your design file. It utilizes a highly recommended modular structure for robust synthesis: sequential state transition logic, fully isolated next-state combinational logic, and clean output decode logic.

```
// =====
// DESIGN UNIT: traffic_light_controller.v
// TYPE: RTL Hardware Description
// ENCODING: IEEE 1364-2001 Verilog Standard
// =====

module traffic_light_controller (
    input wire clk,          // Hardware master clock signal
    input wire rst_n,        // Asynchronous active-low reset signal

    // Outputs representing Main Street and Side Street signal heads
    // Format: [2]=Green, [1]=Yellow, [0]=Red
    output reg [2:0] main_light,
    output reg [2:0] side_light
);

    // State Encoding Parameters (Binary Encoding for ASIC efficiency)
    parameter S0 = 2'b00; // Main: Green | Side: Red
    parameter S1 = 2'b01; // Main: Yellow | Side: Red
    parameter S2 = 2'b10; // Main: Red   | Side: Green
    parameter S3 = 2'b11; // Main: Red   | Side: Yellow

    // State Storage Elements
    reg [1:0] current_state;
    reg [1:0] next_state;

    // Internal timing counter to calculate cycle duration
    reg [3:0] count;

    // -----
    // BLOCK 1: Synchronous/Sequential State & Counter Update
    // Handles asynchronous system reset and sequential ticking.
    // -----
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            current_state <= S0;
            count          <= 4'd0;
        end else begin
            current_state <= next_state;

            // Clear counter on state transition; else increment
            if (current_state != next_state)
                count <= 4'd0;
            else
                count <= count + 1'b1;
        end
    end

    // -----
    // BLOCK 2: Combinational Logic for Next-State Evaluation
    // Configures the duration matrix for each state.
    // -----
    always @(*) begin
        case (current_state)
            S0: begin
                if (count == 4'd5) next_state = S1;
            end
        end case
    end
endmodule
```

```

        else
            next_state = S0;
        end
    S1: begin
        if (count == 4'd2) next_state = S2;
        else
            next_state = S1;
        end
    S2: begin
        if (count == 4'd4) next_state = S3;
        else
            next_state = S2;
        end
    S3: begin
        if (count == 4'd2) next_state = S0;
        else
            next_state = S3;
        end
    default: next_state = S0;
endcase
end

// -----
// BLOCK 3: Combinational Logic for Output Driver Decode
// Converts state vectors cleanly into physical light control lines.
// -----
always @(*) begin
    case (current_state)
        S0: begin
            main_light = 3'b100; // Green
            side_light = 3'b001; // Red
        end
        S1: begin
            main_light = 3'b010; // Yellow
            side_light = 3'b001; // Red
        end
        S2: begin
            main_light = 3'b001; // Red
            side_light = 3'b100; // Green
        end
        S3: begin
            main_light = 3'b001; // Red
            side_light = 3'b010; // Yellow
        end
        default: begin
            main_light = 3'b001; // Fail-safe default
            side_light = 3'b001;
        end
    endcase
end

endmodule

```

4. Functional Verification Testbench

The testbench is non-synthesizable verification code. It drives the clock tree, forces an asynchronous reset down the structural rails, and tracks state updates inside the simulator console.

```

// =====
// VERIFICATION UNIT: tb_traffic_light_controller.v
// TYPE: Behavioral Testbench Environment
// =====

`timescale 1ns/1ps

module tb_traffic_light_controller;

    // Registers to generate driver inputs
    reg clk;
    reg rst_n;

    // Wires to capture design-under-test outputs
    wire [2:0] main_light;
    wire [2:0] side_light;

    // Unit Under Test (UUT) Instantiation via Port Mapping
    traffic_light_controller uut (
        .clk(clk),
        .rst_n(rst_n),
        .main_light(main_light),
        .side_light(side_light)
    );

    // Clock Generator Loop: 10ns Clock Period (5ns High, 5ns Low)
    always begin
        #5 clk = ~clk;
    end

    // Stimulus Pattern Block
    initial begin
        // Initialize lines
        clk = 0;
        rst_n = 0; // Engage active-low reset asset

        // Hold reset active for 20ns to verify initialization states
        #20 rst_n = 1; // Release reset

        // Execute simulation for 200ns to trace full FSM cycle execution
        #200;

        $display("[VERIFICATION] Simulation timeline completed successfully.");
        $finish;
    end

    // Console Logging Block
    initial begin
        $monitor("Time=%0dns | Reset=%b | Main Light(G-Y-R)=%b | Side Light(G-Y-R)=%b",
            $time, rst_n, main_light, side_light);
    end

    // Value Change Dump (VCD) Generation for Waveform Interaction
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_traffic_light_controller);
    end
endmodule

```

```
end
```

```
endmodule
```

5. Step-by-Step EDA Playground Execution Guide

Follow these specific instructions to simulate your design inside the online cloud-based EDA Playground environment:

- 1 **Access the Platform:** Open a modern web browser and navigate to www.edaplayground.com. Register or log in using a standard profile sync (Google or GitHub).
- 2 **Setup Workspace Panes:** On the left side editor column, locate the **Testbench pane** and insert the verification code block. Locate the right-hand column or design tab, and insert the structural Verilog RTL design block.
- 3 **Configure Tools & Simulators:** Look at the left sidebar menu under 'Languages & Tools':
 - Set *Testbench + Design* to **Verilog/SystemVerilog**.
 - Set *Tools & Simulators* dropdown selection to **Icarus Verilog 0.10.0 (or newer)**.
 - Ensure the checkbox option labeled **Open EPWave after run** is fully marked active.
- 4 **Run and Debug:** Click the dark-green **Run** button at the top header menu. Review the console logs printed directly beneath the editor screen. A dedicated browser window showing the simulated waveform timeline (EPWave Viewer) will render automatically, allowing visual trace analysis of logic changes across simulated nanoseconds.